

PONTÍFICA UNIVERSIDADE CATÓLICA DE GOIÁS
DISCIPLINA: PROJETO DE BANCO DE DADOS
CURSO DE CIÊNCIAS DA COMPUTAÇÃO

ARTHUR BARROS DE QUEIROZ

**RELATÓRIO TÉCNICO: IMPLEMENTAÇÃO DO BANCO DE DADOS
FITCONTROL**

Goiânia

2026

ARTHUR BARROS DE QUEIROZ

**RELATÓRIO TÉCNICO: IMPLEMENTAÇÃO DO BANCO DE DADOS
FITCONTROL**

Relatório técnico apresentado como requisito
parcial de avaliação da disciplina de Banco de
Dados.

Goiânia

2026

RESUMO

Este relatório técnico documenta o projeto e a implementação do banco de dados relacional FitControl, uma solução centralizada para o gerenciamento de academias de médio e grande porte. São descritas a arquitetura física do esquema (DDL), as estratégias de integridade referencial adotadas, os scripts de população de dados (DML) com cenários de teste intencional, as consultas SQL para inteligência de negócio, os objetos de banco de dados, views e triggers, e o controle de transações. A abordagem de engenharia empregada prioriza a resiliência, a rastreabilidade financeira e a consistência das prescrições de treino, garantindo uma base sólida para a tomada de decisão gerencial.

Palavras-chave: banco de dados relacional; integridade referencial; SQL; PostgreSQL; gestão de academias.

1 INTRODUÇÃO

O sistema FitControl foi projetado para atuar como uma solução centralizada e robusta para o gerenciamento de academias de médio e grande porte. Este relatório técnico documenta a arquitetura física, as estratégias de integridade referencial e a lógica procedural implementadas no banco de dados.

O objetivo primordial foi criar um esquema que não apenas armazene dados operacionais, mas que garanta a rastreabilidade financeira e a consistência das prescrições de treino, fornecendo uma base sólida para inteligência de negócio.

2 ARQUITETURA DE DADOS: MODELO FÍSICO (DDL)

A arquitetura do FitControl utiliza um esquema relacional normalizado para mitigar a redundância. Priorizou-se o uso de restrições (Constraints) para automatizar a manutenção da integridade. As tabelas foram criadas na seguinte ordem de dependência:

- a) **PLANO**: Tabela mestre que define as modalidades comerciais.
- b) **INSTRUTOR**: Cadastro de profissionais técnicos.
- c) **EXERCICIO**: Catálogo de atividades categorizadas por grupos musculares.
- d) **ALUNO**: Cadastro de clientes. Aplicou-se ON DELETE SET NULL na chave estrangeira do plano, garantindo que o histórico do aluno não seja apagado ao se descontinuar um plano.
- e) **TREINO**: Documento de prescrição. Implementa regra híbrida: ON DELETE CASCADE para o aluno e ON DELETE RESTRICT para o instrutor.
- f) **TREINO_EXERCICIO**: Tabela associativa (N:M) que detalha a execução técnica (séries, repetições e carga).

A seguir, apresenta-se o script DDL completo de criação das tabelas:

```
-- ETAPA 2 - IMPLEMENTAÇÃO FÍSICA: SCRIPT DDL
-- SISTEMA: FITCONTROL -- AUTOR: ARTHUR BARROS DE QUEIROZ

CREATE TABLE PLANO (
    id_plano    INT          NOT NULL,
    nome        VARCHAR(50)  NOT NULL,
```

```

        valor          DECIMAL(6,2) NOT NULL,
        CONSTRAINT PK_PLANO PRIMARY KEY (id_plano)
    );

CREATE TABLE INSTRUTOR (
    id_instrutor INT NOT NULL,
    nome VARCHAR(100) NOT NULL,
    especialidade VARCHAR(50),
    CONSTRAINT PK_INSTRUTOR PRIMARY KEY (id_instrutor)
);

CREATE TABLE EXERCICIO (
    id_exercicio INT NOT NULL,
    nome_exercicio VARCHAR(100) NOT NULL,
    grupo_muscular VARCHAR(50) NOT NULL,
    CONSTRAINT PK_EXERCICIO PRIMARY KEY (id_exercicio)
);

CREATE TABLE ALUNO (
    id_aluno INT NOT NULL,
    nome VARCHAR(100) NOT NULL,
    cpf VARCHAR(11) NOT NULL,
    data_ultima_mensalidade DATE,
    id_plano INT,
    CONSTRAINT PK_ALUNO PRIMARY KEY (id_aluno),
    CONSTRAINT UN_ALUNO_CPF UNIQUE (cpf),
    CONSTRAINT FK_ALUNO_PLANO FOREIGN KEY (id_plano)
        REFERENCES PLANO(id_plano) ON DELETE SET NULL
);

CREATE TABLE TREINO (
    id_treino INT NOT NULL,
    nome_treino VARCHAR(50) NOT NULL,
    data_criacao DATE NOT NULL,
    id_aluno INT NOT NULL,
    id_instrutor INT NOT NULL,
    CONSTRAINT PK_TREINO PRIMARY KEY (id_treino),
    CONSTRAINT FK_TREINO_ALUNO FOREIGN KEY (id_aluno)
        REFERENCES ALUNO(id_aluno) ON DELETE CASCADE,
    CONSTRAINT FK_TREINO_INSTRUTOR FOREIGN KEY (id_instrutor)
        REFERENCES INSTRUTOR(id_instrutor) ON DELETE RESTRICT
);

CREATE TABLE TREINO_EXERCICIO (
    id_treino INT NOT NULL,
    id_exercicio INT NOT NULL,
    series INT NOT NULL,
    repeticoes INT NOT NULL,
    carga INT NOT NULL,
    CONSTRAINT PK_TREINO_EXERCICIO PRIMARY KEY (id_treino, id_exercicio),
    CONSTRAINT FK_TE_TREINO FOREIGN KEY (id_treino)
        REFERENCES TREINO(id_treino) ON DELETE CASCADE,
    CONSTRAINT FK_TE_EXERCICIO FOREIGN KEY (id_exercicio)
        REFERENCES EXERCICIO(id_exercicio) ON DELETE RESTRICT
);

```

3 POPULAÇÃO DE DADOS E TESTES DE INTEGRIDADE (DML)

A estratégia de carga de dados foi desenhada para simular um cenário de produção exaustivo. Além dos registros operacionais padrão, foram inseridos intencionalmente dados com inconsistências para validar os protocolos de auditoria e limpeza (Data Cleaning).

3.1 Análise de Inconsistências Intencionais para Testes

Quadro 1 — Inconsistências intencionais inseridas para fins de teste

Tabela	Registro/ID	Problema Identificado	Objetivo do Teste de Validacao
PLANO	ID 21	Valor negativo (-50,00)	Testar filtros de faturamento e regras de negocio financeiro.
INSTRUTOR	ID 210	Especialidade NULL	Validar consultas de integridade de cadastro profissional.
ALUNO	ID 21	Plano NULL (Inadimplente)	Validar a eficiencia do LEFT JOIN na identificacao de alunos sem vinculo.
ALUNO	ID 22	Data futura (2030)	Detectar erros de digitacao e falta de validacao temporal no front-end.
TREINO_EXERCICIO	ID 514	Series/Repeticoes = 0	Identificar prescricoes de treino incompletas ou invalidas.

Fonte: elaborado pelo autor (2026).

3.2 Scripts de Inserção (DML)

Os scripts de inserção abrangem 22 planos, 21 instrutores, 21 exercícios, 22 alunos, 21 treinos e 21 registros em TREINO_EXERCICIO. Por razões de espaço, apresenta-se a seguir uma amostra representativa dos scripts DML:

```
-- DML_PLANO (amostra)
INSERT INTO PLANO (id_plano, nome, valor) VALUES
  (1, 'Mensal Padrao', 119.90),
  (2, 'Trimestral Bronze', 299.70),
  (3, 'Anual Ouro', 999.00),
  -- ... (22 registros no total, incluindo ID 21 com valor -50.00)
  (21, 'Plano Erro Valor Negativo', -50.00);

-- DML_ALUNO (amostra)
INSERT INTO ALUNO (id_aluno, nome, cpf, data_ultima_mensalidade, id_plano)
VALUES
  (1, 'Arthur Barros', '12345678901', '2026-05-10', 3),
  -- ...
  (21, 'Aluno Sem Plano Inadimplente', '85285285285', NULL, NULL),
  (22, 'Aluno Data Incorreta', '96396396396', '2030-12-31', 1);

-- DML_TREINO_EXERCICIO (amostra com dado invalido)
INSERT INTO TREINO_EXERCICIO
  (id_treino, id_exercicio, series, repeticoes, carga) VALUES
```

```
(501, 100, 4, 10, 50),
(514, 112, 0, 0, 0); -- series e repeticoes zerados (ID 514)
```

4 RELATÓRIOS E INTELIGÊNCIA DE NEGÓCIO

Esta seção apresenta as consultas SQL desenvolvidas para extrair informações críticas que suportam a tomada de decisão gerencial.

4.1 Relatório 1: Visão Geral de Matrículas Ativas

Lista todos os alunos vinculados a planos, servindo como base principal para conferência de chamadas e acessos.

```
SELECT A.id_aluno, A.nome AS nome_aluno, A.cpf,
       P.nome AS nome_plano, P.valor AS valor_mensalidade
FROM   ALUNO A
INNER JOIN PLANO P ON A.id_plano = P.id_plano
ORDER BY A.nome;
```

4.2 Relatório 2: Consolidação de Fichas de Treino

Gera documento completo para o aluno, unindo dados de 5 tabelas para exibir exercícios, cargas e supervisão do instrutor.

```
SELECT T.id_treino, A.nome AS nome_aluno, I.nome AS nome_instrutor,
       T.nome_treino, E.nome_exercicio, TE.series, TE.repeticoes, TE.carga
FROM   TREINO T
INNER JOIN ALUNO A          ON T.id_aluno      = A.id_aluno
INNER JOIN INSTRUTOR I      ON T.id_instrutor  = I.id_instrutor
INNER JOIN TREINO_EXERCICIO TE ON T.id_treino   = TE.id_treino
INNER JOIN EXERCICIO E      ON TE.id_exercicio = E.id_exercicio
ORDER BY A.nome, T.nome_treino;
```

4.3 Relatório 3: Análise de Faturamento Mensal Estimado

Visão executiva que agrupa alunos por plano para projetar a receita mensal da unidade.

```
SELECT P.nome AS nome_plano,
       COUNT(A.id_aluno) AS total_alunos_matriculados,
       SUM(P.valor)      AS faturamento_estimado_mensal
FROM   PLANO P
INNER JOIN ALUNO A ON P.id_plano = A.id_plano
GROUP BY P.nome, P.valor
ORDER BY total_alunos_matriculados DESC;
```

4.4 Relatório 4: Gestão de Carga de Trabalho dos Instrutores

Identifica instrutores com alta demanda operacional (mais de 2 treinos prescritos), permitindo o balanceamento da equipe.

```
SELECT I.id_instrutor, I.nome AS nome_instrutor, I.especialidade,
       COUNT(T.id_treino) AS total_treinos_prescritos
FROM   INSTRUTOR I
INNER JOIN TREINO T ON I.id_instrutor = T.id_instrutor
GROUP BY I.id_instrutor, I.nome, I.especialidade
HAVING COUNT(T.id_treino) > 2
ORDER BY total_treinos_prescritos DESC;
```

4.5 Relatório 5: Alunos de Alta Performance (Outliers)

Utiliza subconsulta correlacionada para identificar alunos cujas cargas superam a média global da academia.

```
SELECT DISTINCT A.nome AS nome_aluno, T.nome_treino,
               E.nome_exercicio, TE.carga
FROM   ALUNO A
INNER JOIN TREINO T           ON A.id_aluno      = T.id_aluno
INNER JOIN TREINO_EXERCICIO TE ON T.id_treino    = TE.id_treino
INNER JOIN EXERCICIO E       ON TE.id_exercicio = E.id_exercicio
WHERE  TE.carga > (SELECT AVG(carga) FROM TREINO_EXERCICIO)
ORDER BY TE.carga DESC;
```

5 AUDITORIA E QUALIDADE DE DADOS (DATA CLEANING)

Para garantir a higiene dos dados e a confiabilidade dos relatórios financeiros, as consultas a seguir atuam como ferramentas de auditoria, sinalizando registros que fogem aos padrões de qualidade exigidos pelo FitControl.

```
-- CONSULTA 6: Alunos Inadimplentes ou Sem Vinculo de Plano
SELECT A.id_aluno, A.nome, A.cpf, A.data_ultima_mensalidade, A.id_plano
FROM   ALUNO A
LEFT JOIN PLANO P ON A.id_plano = P.id_plano
WHERE  A.id_plano IS NULL OR A.data_ultima_mensalidade IS NULL;

-- CONSULTA 7: Erros de Precificacao (Valores Negativos ou Zerados)
SELECT id_plano, nome AS nome_plano, valor
FROM   PLANO WHERE valor <= 0;

-- CONSULTA 8: Cadastro Profissional Incompleto
SELECT id_instrutor, nome, especialidade
FROM   INSTRUTOR WHERE especialidade IS NULL;

-- CONSULTA 9: Exercicios Obsoletos (Sem Uso em Fichas) - RIGHT JOIN
SELECT E.id_exercicio, E.nome_exercicio, E.grupo_muscular
```



```

FROM    TREINO_EXERCICIO TE
RIGHT JOIN EXERCICIO E ON TE.id_exercicio = E.id_exercicio
WHERE   TE.id_exercicio IS NULL;

-- CONSULTA 10: Integridade Tecnica das Fichas (Series/Repeticoes Zeradas)
SELECT T.id_treino, T.nome_treino, E.nome_exercicio,
       TE.series, TE.repeticoes, TE.carga
FROM    TREINO T
INNER JOIN TREINO_EXERCICIO TE ON T.id_treino = TE.id_treino
INNER JOIN EXERCICIO E          ON TE.id_exercicio = E.id_exercicio
WHERE   TE.series = 0 OR TE.repeticoes = 0;

```

6 OBJETOS DE BANCO DE DADOS: VIEWS E TRIGGERS

6.1 Visões (Views)

As visões foram criadas para abstrair a complexidade das junções (JOINS) para o usuário final ou para dashboards de front-end, oferecendo interfaces simplificadas e reutilizáveis:

```

-- VW_FICHAS_TREINO_DETALHADAS: abstrai junção de 5 tabelas
CREATE VIEW VW_FICHAS_TREINO_DETALHADAS AS
SELECT T.id_treino, A.nome AS nome_aluno,
       I.nome AS nome_instrutor, T.nome_treino,
       E.nome_exercicio, TE.series, TE.repeticoes, TE.carga
FROM    TREINO T
INNER JOIN ALUNO A              ON T.id_aluno = A.id_aluno
INNER JOIN INSTRUTOR I          ON T.id_instrutor = I.id_instrutor
INNER JOIN TREINO_EXERCICIO TE  ON T.id_treino = TE.id_treino
INNER JOIN EXERCICIO E          ON TE.id_exercicio = E.id_exercicio;

-- VW_ALUNOS_INCONSISTENTES: visao de dados sujos para cobrança
CREATE VIEW VW_ALUNOS_INCONSISTENTES AS
SELECT A.id_aluno, A.nome AS nome_aluno, A.cpf, A.data_ultima_mensalidade
FROM    ALUNO A
LEFT JOIN PLANO P ON A.id_plano = P.id_plano
WHERE   A.id_plano IS NULL OR A.data_ultima_mensalidade IS NULL;

```

6.2 Automação e Auditoria (Triggers)

Implementou-se uma trigger em PL/pgSQL para monitorar alterações em datas de pagamento, registrando o usuário do sistema (CURRENT_USER) e o timestamp da operação, protegendo a academia contra fraudes financeiras:

```

-- Tabela de auditoria
CREATE TABLE LOG_RENOVACAO_ALUNO (
    id_log          SERIAL          NOT NULL,
    id_aluno        INT             NOT NULL,
    data_alteracao  TIMESTAMP       NOT NULL,
    usuario_sistema VARCHAR(50)     NOT NULL,
    antiga_data_pagamento DATE,
    nova_data_pagamento DATE,
    CONSTRAINT PK_LOG_RENOVACAO PRIMARY KEY (id_log)

```

```

);

-- Funcao do Trigger
CREATE OR REPLACE FUNCTION fn_auditoria_pagamento_aluno()
RETURNS TRIGGER AS $$
BEGIN
    IF NOT (OLD.data_ultima_mensalidade IS NOT DISTINCT FROM
            NEW.data_ultima_mensalidade) THEN
        INSERT INTO LOG_RENOVACAO_ALUNO
            (id_aluno, data_alteracao, usuario_sistema,
             antiga_data_pagamento, nova_data_pagamento)
        VALUES
            (NEW.id_aluno, NOW(), CURRENT_USER,
             OLD.data_ultima_mensalidade, NEW.data_ultima_mensalidade);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Criacao do Trigger
CREATE TRIGGER TRG_AUDITORIA_PAGAMENTO_ALUNO
AFTER UPDATE ON ALUNO
FOR EACH ROW EXECUTE FUNCTION fn_auditoria_pagamento_aluno();

```

7 CONTROLE DE TRANSAÇÕES E CONCORRÊNCIA

O controle transacional garante a atomicidade das operações críticas do FitControl, prevenindo corrupção de dados em cenários de erro. Três cenários foram implementados e testados:

- a) **Cenário A (COMMIT):** sucesso em matrícula em lote, consolidando as alterações permanentemente.
- b) **Cenário B (ROLLBACK por Integridade):** proteção contra violação de integridade referencial.
- c) **Cenário C (ROLLBACK de Segurança):** recuperação de DELETE global acidental sem cláusula WHERE.

```

-- CENARIO A: Matricula em lote com COMMIT (sucesso)
BEGIN;
INSERT INTO ALUNO (id_aluno, nome, cpf, data_ultima_mensalidade, id_plano)
VALUES (100, 'Arthur Teste Commit', '00011122233', '2026-06-10', 1);
COMMIT;

-- CENARIO B: Violacao de FK com ROLLBACK (erro proposital)
BEGIN;
INSERT INTO ALUNO (id_aluno, nome, cpf, data_ultima_mensalidade, id_plano)
VALUES (101, 'Aluno Erro Rollback', '00099988877', '2026-06-10', 1);
ROLLBACK;

-- CENARIO C: DELETE em massa acidental com ROLLBACK de seguranca
BEGIN;
DELETE FROM PLANO;

```

```

ROLLBACK;

-- Consultas de validacao
SELECT * FROM ALUNO WHERE id_aluno = 100; -- deve retornar 1 linha (COMMIT)
SELECT * FROM ALUNO WHERE id_aluno = 101; -- deve retornar 0 linhas
(ROLLBACK)
SELECT * FROM PLANO; -- tabela integra apos ROLLBACK
SELECT * FROM LOG_RENOVACAO_ALUNO; -- log do Trigger

```

8 CONSIDERAÇÕES FINAIS

A implementação do banco de dados FitControl demonstra uma abordagem de engenharia de software focada em resiliência. Através do uso rigoroso de Chaves Primárias, Estrangeiras e regras de propagação de exclusão, garantiu-se a integridade referencial em todos os níveis do modelo relacional.

A camada de auditoria via triggers e as visões de inteligência de negócio transformam este banco de dados em uma ferramenta estratégica para a gestão da academia, assegurando que cada informação seja rastreável e útil para o crescimento da operação. Os cenários de teste com dados intencionalmente inconsistentes validaram a robustez das regras de negócio implementadas.

REFERÊNCIAS

- ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.
- ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de banco de dados. 6. ed. São Paulo: Pearson, 2011.
- POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 16 Documentation. Disponível em: <https://www.postgresql.org/docs/16/>. Acesso em: 10 jun. 2026.
- SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. Sistema de banco de dados. 7. ed. Rio de Janeiro: GEN LTC, 2020.